The Free Flip: Directional Composition Confounds Attribution in Paired Training Data

Anonymous Author(s)

Abstract

A model can be trained to obfuscate code, or to deobfuscate it. The same pairs of programs serve both tasks, because training the second direction only requires swapping which half of each pair is the input. Compilation and decompilation, and translation between two languages, work the same way. This complicates attribution. A method that adds training instances also changes how many examples of each direction the model sees. If such a method is credited with improving reverse performance, the gain may come from the reverse examples it added rather than from the method itself, and the controls usually reported do not distinguish the two, since instance count, token count, optimizer steps and compute can all be matched while the number of reverse examples differs. We measure this in one case. Fine-tuning a model on the forward direction removes the reverse one, and recent work reports that a contrastive objective repairs the loss without using any reverse examples [Nikiema et al., 2025]. That objective also adds instances, so we ran the 2×2 that holds the direction mix fixed. On our reading of the published objective it contributes -0.2 percentage points (95% confidence interval $[-0.7, +0.3]$), against $+30.9$ for the direction of the data, which attributes the reported effect to the data the method supplies. The resulting test is general and needs no new data. For any method that adds instances to paired data, train the arm that supplies the same mix of directions and none of the method. Here that arm also repairs the failure, reaching 32.8% reverse accuracy at unchanged training cost. Two limits apply. The repair works only where the rewrite can be undone, and fine-tuning costs about five points of held-out general ability at any direction mix, so it is free relative to the forward-only baseline rather than to the untouched model.

1 Introduction

Paired data carries a direction. Obfuscating a program rewrites it so that it behaves identically but is hard to read, by renaming `total_price` to `v_a3f2` or replacing control flow with a dispatch loop. A model can be trained to perform that rewrite, which we call the **forward** direction, or to undo it, the **reverse** direction, which is the one practitioners want (Figure 1). The two tasks use the same pairs of programs. Training the reverse direction only requires swapping which half of each pair is the input. Compilation and decompilation, minification and beautification, and translation between two languages all work this way.

That convenience creates an attribution problem. *A method that adds training instances also changes how many examples of each direction the model sees.* Suppose such a method is credited with improving reverse performance, and the instances it adds include reverse examples. The gain could come from that added exposure rather than from the method, and the controls a reader normally sees will not distinguish the two, because instance count, token count, optimizer steps and compute can all be matched exactly while the number of reverse examples differs.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Attributing Model Behavior at Scale (ATTRIB).

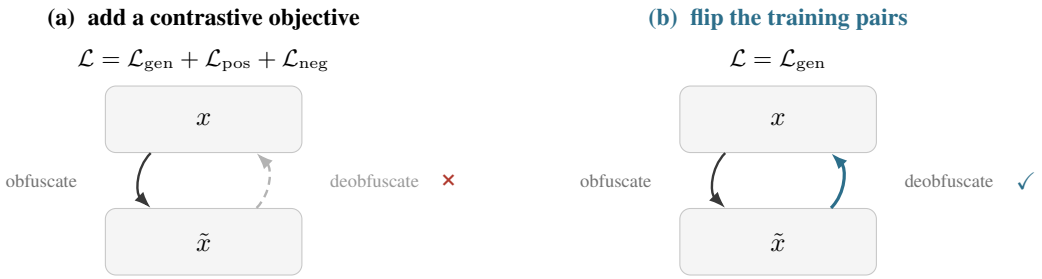

Figure 1: The attribution this paper tests. Both panels fine-tune the same model on the same obfuscation pairs. **(a)** Contrastive Fine-Tuning augments the generation loss with two equivalence-judgement terms and is credited with restoring the reverse direction. In our experiments it does not. Reverse success stays at zero and falls below the untouched model. **(b)** Keeping the plain generation loss and reading half of the same pairs backwards restores the reverse direction at identical training cost. The recovery comes from the direction of the data, not from the objective.

The phenomenon this paper is about is that fine-tuning on one of those directions removes the other. At 7B, forward-only fine-tuning takes reverse success from 12.9% to 0.0%. The loss is also disproportionate rather than a general degradation, which is what distinguishes it from ordinary catastrophic forgetting. The same model gives up about five points of held-out general coding ability, .662 on MBPP+ against .714 untouched, while the reverse direction is gone completely, so what fine-tuning removed was specifically the inverse of the mapping it was taught. A practitioner who fine-tunes a model to apply a transformation should expect to lose the ability to undo it, and to see almost none of that loss in a general benchmark.

The reported remedy, and why it is an attribution problem. Nikiema et al. [2025] document this failure, name it *cognitive specialization*, and propose **Contrastive Fine-Tuning** (CFT) as a remedy. CFT adds two equivalence-judgement terms to the generation loss (Figure 1a), training the model to say whether two programs are semantically equivalent alongside learning to obfuscate. They report that this recovers 39–52% reverse success *without any reverse training data*, which is what makes the result striking, because it suggests the capability is unlocked by how the model is trained rather than by what it is shown.

CFT has exactly this structure. It varies two factors together, adding an objective and adding the equivalence-judgement instances that objective requires, which are data the forward-only baseline does not see. This is the ordinary form of an auxiliary-task method rather than an oversight particular to that work. The original paper's evaluation section does define a Bidirectional Fine-Tuning condition, forward generation plus reverse deobfuscation, but reports no result for it, so we run that comparison here.

A test that needs no new data. The direction of the data accounts for essentially the whole effect and the objective for none of it (Figure 1b). The general form of the check is simple. *For any method that adds instances to paired data, train the arm that supplies the same mix of directions and none of the method.* That arm costs nothing to build, because the reverse examples are the forward examples with their halves swapped, so there is nothing to collect and nothing to annotate. If it reproduces the gain, the gain belongs to the data the method supplied. Here it does. Half forward and half reversed pairs, at identical cost to forward-only training, raise reverse success from 0.0% to 32.8%, while our reimplementation of the objective reaches 0.1%. The same arm also repairs the failure, so running the check is worth doing whatever it shows. Two qualifications bound it. Reading pairs backwards helps only where the mapping can in principle be inverted, and it does not restore what fine-tuning costs in general coding ability, which on a held-out benchmark is about five points for every arm we trained, the forward-only baseline included.

2 Setup

Programs and rewrites. We use Python functions that come with executable test inputs, drawn from APPS [Hendrycks et al., 2021], CRUXEval [Gu et al., 2024] and HumanEval [Chen et al., 2021]. Each function is rewritten under five obfuscating transforms. Three rewrite identifiers, by

renaming them to misleading names (`L1b`), to random strings such as `v_a3f2` (`L1r`), or to single letters with type annotations stripped (`L2`). Two rewrite structure, by flattening control flow into a dispatch loop (`S1`) or inserting always-true tests and unreachable code (`S2`). That split matters later, because only the structural rewrites can be undone. Full definitions are in Appendix I. Every rewrite is applied to the original function on its own, never stacked, and is kept only if the rewritten program still produces the original outputs.

How we score a trial. For one trial let x be the original readable program, $\tilde{x}$ its obfuscated form, and $\hat{y}$ the model's output. Four quantities build the criteria. $\mathrm{parses}(\hat{y})$ is 1 when the output is non-empty and parses in the target language. $S(a, b) \in [0, 1]$ is CodeBLEU similarity between two programs [Ren et al., 2020]. $R(a) \in [0, 1]$ is readability, a weighted sum of identifier meaningfulness (0.55) with identifier length, line length and nesting depth (0.15 each), and it returns 0 for a non-parsing output, since the readability of a non-program is undefined and 0 is the only resolution that cannot manufacture a success. $E(\hat{y})$ is 1 when the output executes and reproduces x's outputs on every test input that ships with the program. Appendix H restates these as a table.

In the forward direction the model must emit an obfuscated program, and we score it by whether that program still computes the original outputs, so forward success is $E(\hat{y})$.

For the reverse direction we report two criteria, because the published one turns out to be permissive. The **published criterion** is the test used by Nikiema et al. [2025], which asks that the output be textually dissimilar to the obfuscated input and about as readable as the original,

$$\mathrm{Pub}(\hat{y}) \;=\; \mathrm{parses}(\hat{y}) \;\wedge\; \big[\, S(\hat{y}, \tilde{x}) < 0.4 \,\big] \;\wedge\; \big[\, R(\hat{y}) \geq R(x) - 0.1 \,\big], \tag{1}$$

where the thresholds are that paper's values and the parses conjunct is ours. The guard is not optional, because without it the first inequality is satisfied by any reply that merely differs from the input, and a run of fixed placeholder strings scored 17–25% "reverse success" before we added it. It can only lower a reported number, never inflate one.

Strict success, our headline metric, additionally requires that the recovered program run correctly,

$$\mathrm{Strict}(\hat{y}) \;=\; \mathrm{Pub}(\hat{y}) \;\wedge\; E(\hat{y}). \tag{2}$$

A deobfuscation that does not run has recovered nothing. We track **echoing**, $\mathrm{Echo}(\hat{y}) = \mathbf{1}[\hat{y} = \tilde{x}]$, separately rather than letting replies that return the input count towards accuracy.

One substitution matters for Eq. (1). The original study measures readability with the model of Scalabrino et al. [2018], which we do not have, so R is the proxy in Table 6. We checked what that costs by recomputing every result while varying the readability treatment, including dropping the term entirely, and strict success moves by at most 1.3 pp on any trained arm against an effect of 33 points (Appendix B). Execution and the similarity bound do essentially all the deciding.

What we compare. Every model in this paper is the same base model with a small set of adapter weights trained on top, using LoRA [Hu et al., 2022], and every one uses an identical recipe (rank 32, three epochs, learning rate 10^{-4}, batch 64, seed 17). The only thing that differs between them is the mixture of training examples, so no difference between them can come from hyperparameters. We call each mixture an **arm**; Table 4 in Appendix A defines all eight.

Evaluation and uncertainty. We decode greedily on 300 held-out programs, using only programs that every transform succeeded on so that no two cells are scored on different populations. All intervals are 95% confidence intervals from a bootstrap that resamples *programs* rather than individual test cases, because several test cases share a program and are therefore correlated. We write **pp** for percentage points. Greedy decoding is not bitwise repeatable here, since batching changes which token wins a near-tie, so two passes over the same programs differ on 6 to 8% of generations and move a headline rate by about 0.2 pp (Appendix B). Every table comes from a single pass, we compare arms only within a pass, and we never quote a cross-pass difference below 0.5 pp.

3 Decomposing the objective and the data direction

Table 1 varies the two factors CFT bundles together. One factor is whether the equivalence-judgement objective is present. The other is whether the training data contains the reversed pairs. All four cells use the same 300 programs and the same 18 000 graded attempts. This section and the dose response are at 1.5B. Three of the four cells also exist at 7B and are in Table 2, so both main

Table 1: The 2×2 experiment. Qwen2.5-Coder-1.5B [Hui et al., 2024], strict reverse success in per cent, 300 programs and 18 000 graded attempts, all cells scored on the same programs. The right-hand panel decomposes the four cells into the effect of each factor.

	forward only	+ reversed pairs
no judgement objective	`sft` 0.3	`flip` **31.4**
with judgement objective	`cft` 0.3	`cftflip` **31.1**

effect of	estimate (pp)	95% CI
reversing the data	$\mathbf{+30.9}$	$[+29.3, +32.6]$
the objective	$\mathbf{-0.2}$	$[-0.7, +0.3]$
the two interacting	-0.3	$[-1.3, +0.7]$

Table 2: Qwen2.5-Coder-7B, 300 programs, 21 000 graded attempts. *Forward* is scored by execution. *Reverse (strict)* requires the published criterion and that the output run correctly. *Echo* is how often the model returns its input unchanged.

arm	forward	**reverse (strict)**	reverse (published crit.)	echo
`base` *(untouched)*	.929	**.129**	.233	.073
`sft` *(forward only)*	.971	**.000**	.004	.182
`fwd2x` *(2× epochs)*	.969	**.001**	.019	.071
`cft` *(published method)*	.975	**.001**	.002	**.293**
`rev` *(reverse only)*	.927	**.329**	.351	.000
`flip`	.971	**.335**	.353	.000
`mix50` *(same cost as `sft`)*	.961	**.328**	.356	.000

effects reproduce at the scale the abstract quotes; only the interaction term rests on 1.5B, because the fourth cell would have cost a further 8.8 GPU-hours on a shared machine.

The objective's contribution is not merely too small to detect. It is bounded under one percentage point in either direction whether or not reversed data is present, with an interaction that also spans zero. Adding it to forward-only training moves reverse success by -0.1 pp $[-0.5, +0.3]$, and adding it on top of reversed data by -0.3 pp $[-1.2, +0.5]$, against roughly 31 points for reversing the data. Whatever recovers the reverse direction, it is not the objective.

Training longer does not explain it either. `fwd2x` trains forward-only for twice as many optimizer steps and reaches 0.6%, which is $+0.3$ pp over `sft` with an interval of $[-0.2, +0.8]$.

How much reversed data is needed, and it is much less than half. Appendix D replaces forward examples with reversed ones in increasing amounts. Replacing only 5% of them already recovers 26.1%, against 0.3% for forward-only, which is 85% of what a 50% share achieves. The remaining climb is real but small, worth $+4.5$ pp $[+3.2, +5.8]$, and it has flattened by the time a quarter of the data is reversed. So the intervention is not a matter of data volume. A small amount of exposure to the reverse direction captures most of the effect.

4 The budget-matched arm, which is also the remedy

The obvious objection to Table 1 is that `flip` sees twice as many examples as `sft`. `mix50` answers it by *replacing* half the forward examples rather than adding to them, so it trains on the same 7 383 examples for the same 346 optimizer steps. Two quantities matter separately. **Sequence tokens** are everything the model reads, which sets compute; **supervised tokens** are the subset the loss is computed on, which sets how much it is taught. `mix50` uses $1.05\times$ the sequence tokens of `sft` and $0.71\times$ the supervised tokens, so it is taught *less*, not more.

`mix50` reaches $+32.8$ pp over `sft` $[+31.3, +34.4]$ while costing no more on any axis a practitioner pays for, and `flip`, which does spend $2.09\times$ the sequence tokens, buys only $+0.7$ pp more $[-0.3, +1.8]$, an interval spanning zero. The extra compute is not what produces the capability. The exposure is. The published method fares worst here, spending $2.65\times$ the compute of forward-only training to add 2% more supervised signal, because each equivalence judgement puts two whole

Table 3: pass@1, no scorer errors in any cell. HumanEval+ is 164 problems, of which 74 are in our training split; MBPP+ is 399 and is held out completely. The grouping visible in the 7B HumanEval+ column does not survive on data the arms never saw.

	HumanEval+ *(74/164 seen)*		MBPP+ *(0/399 seen)*
arm	1.5B	7B	**7B**
`base` *(untouched)*	**.646**	**.805**	**.714**
`sft` *(forward only)*	.329	.817	.662
`fwd2x` *(forward, 2× epochs)*	n/a	.805	.689
`cft` *(forward + judgements)*	.366	.799	.664
`rev` *(reverse only)*	.585	.799	.654
`flip`	.512	.744	.652
`mix50`	.469	.732	.659

programs in the prompt and supervises one word. The echo column indicates what it does train. `cft` returns its input unchanged on 29.3% of trials against 18.2% for `sft` and 7.3% untouched, so it does not merely fail to restore the reverse direction, it moves the model toward copying.

The replication fails in the direction that matters. Our `cft` arm reaches 0.1% strict reverse success at 7B, against a reported 39–52%. The gap is not merely a shortfall, since `cft` lands 12.8 pp *below the untouched model* $[-14.8, -10.9]$, which itself scores 12.9%, and forward-only training lands equally far below. Both recipes *remove* a capability the base model already had, and the contrastive one does not restore it. We report this at 7B because that is the size at which the original study's open-source results sit. Our corpus is Python and smaller than theirs, and our `cft` arm is a reconstruction. The original states its objective as three instance pools with counts rather than as a loss, never using the words InfoNCE, margin, cosine or embedding, and two of its three headline CFT results come from provider fine-tuning APIs that take only supervised input→output pairs and cannot express an embedding-space loss at all, so joint next-token cross-entropy over the pools is the reading its own description supports (Appendix B). This is a failure to reproduce in our setting rather than a verdict on contrastive training, and the positive result rests on neither, since `mix50` and `flip` are ordinary supervised fine-tuning.

5 Two limits on the remedy

It only works where the transformation has an inverse. The headline 33% averages five transforms that behave nothing alike, and the average hides a ceiling. On the two structural transforms the bidirectional arms reach about 80%, while on the three renaming transforms *every* arm sits at or below 2.3%, including `rev`, which trains on nothing but deobfuscation (Appendix J). The cause is the task, not the data. Once `total_price` has become `v_a3f2` the original name is absent from the input, so no quantity of reverse supervision recovers it, and reverse training should not be expected to help on transforms that destroy information rather than rearrange it. The published criterion is uninformative here as well (Appendix M).

It costs general coding ability, and the cost belongs to fine-tuning rather than to bidirectionality. On forward accuracy the flip is genuinely free, matching `sft` on every transform (Appendix K). On general ability nothing is free, and we measure it twice because one of the two probes is contaminated. **HumanEval+** [Liu et al., 2023] is 164 problems scored by execution, where **pass@1** is the fraction solved first try, and HumanEval is one of our corpus's three sources, so 74 of those 164 sit in our training split. **MBPP+** is 399 problems our corpus excludes entirely, 0 of 399. Only the second supports a comparison against the untouched model.

On the held-out probe the arms are not distinguishable from one another, and all of them sit below the untouched model. Every trained arm except `fwd2x` is significantly under it, by 2.5 to 6.3 points, and no arm-against-arm contrast resolves. `mix50` minus `sft` is -0.3 pp $[-4.0, +3.5]$, and the both-directions minus one-direction contrast that HumanEval+ puts at 6 to 7 points is -1.2 pp $[-3.8, +1.5]$ on MBPP+. Pooled over the six trained arms the cost is 5.1 pp $[-8.0, -2.1]$ whatever the mixture. The grouping in the HumanEval+ column is an artifact of what those arms had already seen. `sft` scores 1.2 points *above* the untouched model on the benchmark 74 of whose problems it trained on, and 5.3 points below it $[-9.0, -1.5]$ on the benchmark it never saw. Intervals are

paired bootstraps over the 399 tasks, and the sign of every conclusion here is confirmed by an exact McNemar test on the discordant tasks.

Two consequences follow, in opposite directions. The remedy becomes cheaper, since bidirectional data costs no more than the forward-only training a practitioner would run anyway, so the flip is free against the baseline that matters. But forward-only tuning does not leave general ability untouched; it costs about five points. What survives is the disproportion. Reverse success goes to zero, a total loss, while general ability gives up a fourteenth of its value. Two caveats. At 1.5B forward-only training is instead the most damaging arm, which reads as ordinary overfitting to a narrow task. And the HumanEval+ cells predate our per-task logging, so that column carries point estimates only.

6 Related work

Directionality and the reversal curse. The nearest literature to our phenomenon is the reversal curse. Berglund et al. [2024] show that a model trained on "A is B" does not thereby learn "B is A", and Golovneva et al. [2024] cure much of that failure with reverse training, in which strings are additionally presented reversed, evaluated in both data-matched and compute-matched settings. Our budget-matched arm is the same manoeuvre, and their result is a reason to expect ours.

What we report is a different failure with the same symptom. The reversal curse is a failure to *generalize* to a direction never trained, so the capability was never present. Here the untouched model can already work in reverse, scoring 12.9% at 7B, and forward-only fine-tuning *removes* it, taking it to 0.0%. Destroying an ability the model had is not the same event as failing to acquire one, and it is caused by the fine-tuning a practitioner chooses to run. The loss is also disproportionate, costing general coding ability about five points where the reverse direction goes entirely, whereas the reversal curse describes a limit on what a training distribution can teach rather than damage to an existing skill. The inverse here is semantic too, undoing a program transformation rather than reordering tokens or entities.

Deobfuscation. Nikiema et al. [2025] is the study replicated here. A larger literature trains models to recover source from a transformed form, whether obfuscated, compiled or decompiled [Roziere et al., 2021, Tan et al., 2024, Hu et al., 2026], and our setting differs in where the reverse supervision comes from, since we re-read pairs that already exist. Guzmán Lorenzo [2026] finds that misleading identifiers survive deobfuscation almost always, agreeing with our renaming ceiling. All arms are LoRA adapters [Hu et al., 2022].

7 Conclusion

No standard budget control holds the mix of directions fixed, so any method that adds instances to paired data changes it. Here that change accounts for a published effect in full. An objective credited with recovering 39 to 52% of a lost capability contributes -0.2 pp once the direction mix is held constant, against $+30.9$ pp for the mix itself. We do not claim that auxiliary-task results on paired data are generally directional artifacts. We report that the confound is measurable, cheap to test, and to our knowledge untested, and that a factor credited with an effect should be compared against the ablation its own instances make available.

A second finding is independent of the attribution claim. Fine-tuning on one direction of an invertible transformation removes the other, and the loss is disproportionate. Reverse success falls to zero while held-out general ability declines by about five points, so a general benchmark registers a small fraction of the change. It is also cheap to prevent, since the data for the missing direction is the data already in hand read backwards. Our study is Python at two sizes of one model family, with the load-bearing arms replicated in JavaScript at the smaller size, rather than the original's Java; Appendix E lists the limitations in full.

Future work. The other invertible pairs offer a direct test of whether the loss and the repair transfer, and we have not run it. Measurement needs attention too. At 7B a general benchmark whose problems were partly in the training split moved $+1.2$ points while the reverse direction went to zero, and a held-out one moved -5.3, so probes paired to the trained transformation, and probes the model has demonstrably not seen, both appear necessary to catch this class of loss.

References

Lukas Berglund, Meg Tong, Maximilian Kaufmann, Mikita Balesni, Asa Cooper Stickland, Tomek Korbak, and Owain Evans. The reversal curse: LLMs trained on "A is B" fail to learn "B is A". In *International Conference on Learning Representations (ICLR)*, 2024.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Olga Golovneva, Zeyuan Allen-Zhu, Jason Weston, and Sainbayar Sukhbaatar. Reverse training to nurse the reversal curse. *arXiv preprint arXiv:2403.13799*, 2024.

Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. CRUXEval: A benchmark for code reasoning, understanding and execution. *arXiv preprint arXiv:2401.03065*, 2024.

Luis Guzmán Lorenzo. Poisoned identifiers survive LLM deobfuscation: A case study on claude opus 4.6. *arXiv preprint arXiv:2604.04289*, 2026.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Li Hu, Xiuwei Shang, Jieke Shi, Shaoyin Cheng, Junqi Zhang, Gangyang Li, Zhou Yang, Weiming Zhang, and David Lo. Can LLMs deobfuscate binary code? a systematic analysis of large language models into pseudocode deobfuscation. *arXiv preprint arXiv:2604.08083*, 2026.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Serge Lionel Nikiema, Jordan Samhi, Micheline Bénédicte Moumoula, Albérick Euraste Djiré, Abdoul Kader Kaboré, Jacques Klein, and Tegawendé F. Bissyandé. Using contrastive learning to improve two-way reasoning in large language models: The obfuscation task as a case study. *arXiv preprint arXiv:2509.05553*, 2025.

Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. CodeBLEU: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297*, 2020.

Baptiste Roziere, Marie-Anne Lachaux, Marc Szafraniec, and Guillaume Lample. DOBF: A deobfuscation pre-training objective for programming languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Simone Scalabrino, Mario Linares-Vásquez, Rocco Oliveto, and Denys Poshyvanyk. A comprehensive model for code readability. *Journal of Software: Evolution and Process*, 30(6), 2018.

Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. LLM4Decompile: Decompiling binary code with large language models. In *Findings of the Association for Computational Linguistics (EMNLP Findings)*, 2024.

Appendix

A The training mixtures

Every arm is the same base model under the same LoRA recipe, so any difference between two rows is a difference of training data alone. `cftflip` exists only at 1.5B.

Table 4: The training mixtures compared in this paper. Only the mixture differs; the model, recipe and budget are otherwise identical. `mix50` *replaces* forward examples rather than adding to them, which is what makes it cost the same as `sft`.

arm	training examples	why it is in the experiment
`base`	none	the untouched control the original study omits
`sft`	forward only	the original study's forward-only baseline
`cft`	forward + equivalence judgements	our reimplementation of the published method
`flip`	forward + the same pairs reversed	the declared but unreported baseline
`mix50`	half forward, half reversed	the decisive arm, cost-matched to `sft` with *less* supervision
`rev`	reverse only	how well reverse can be done at all
`fwd2x`	forward only, twice the epochs	rules out "it simply trained longer"
`cftflip`	judgements + both directions	the fourth cell of the 2×2

B Reimplementation fidelity, deviations, and the determinism floor

The original objective is instance-level. (i) Its three terms are described only as instance pools with counts ("30 000 instances, 10 000 each for positive classification, negative classification, and obfuscation task generation"); the words InfoNCE, margin, cosine, temperature and embedding do not appear in the paper. (ii) Two of its three headline CFT numbers come from models tuned through provider fine-tuning APIs, which accept only supervised input→output text pairs, so a custom embedding-space loss is not expressible there. (iii) Open-source models are tuned with plain LoRA and no modified objective. We therefore realise $\mathcal{L}_{\text{pos}} + \mathcal{L}_{\text{neg}} + \mathcal{L}_{\text{gen}}$ as joint next-token cross-entropy over the union of the three pools, which is what a sum of per-task losses over balanced datasets is.

Readability is a proxy, and the choice does not matter. Eq. (1) tests whether an output is about as readable as the original, and the original study measures that with Scalabrino et al. [2018]. We do not have that model, so R is the weighted proxy of Table 6, calibrated by measurement rather than guessed, separating clean code from fully minified code at 8% false positives and 89% detection over 400 programs. Because a substituted measure inside a reimplemented criterion is exactly the kind of deviation that can quietly carry a result, we measured its influence directly by recomputing the 7B reverse cells under four treatments of the readability conjunct.

	strict reverse success (%)			
arm	tolerance 0.1 (reported)	term removed	tolerance 0.0	tolerance 0.3
`base`	12.9	13.5	10.6	13.5
`sft`	0.0	0.0	0.0	0.0
`cft`	0.1	0.1	0.0	0.1
`fwd2x`	0.1	0.1	0.0	0.1
`rev`	32.9	32.9	32.1	32.9
`flip`	33.5	33.6	32.3	33.5
`mix50`	32.8	33.0	31.9	33.0

Removing the readability conjunct altogether changes no arm by more than 0.6 pp and changes the `mix50` minus `sft` contrast by 0.2 pp. Across all four treatments the spread is at most 1.3 pp on any trained arm and 2.9 pp on the untuned model, against a headline effect of 33 points. The reason is visible in the criterion itself. Once an output has to parse, be textually far from the obfuscated input, and execute correctly, it is essentially always at least as readable as the original on any reasonable measure, so the conjunct is the deciding factor on only 0.2% of trials for `flip` and `mix50` and 1.0% for `base`. Substituting the readability model therefore cannot account for our conclusions. The one number where the treatment has real leverage is the untuned model's *published-criterion* rate, which ranges from 17.6% to 24.3% across the four settings, and we place no claim on that quantity.

What the original leaves open, and what we chose. The paper gives the generation prompt verbatim but specifies *no prompt and no target format* for the two auxiliary tasks, deferring to supplementary material that is not public. The single-token YES/NO target is therefore ours. That choice

is what produces the token shares below, so they describe our realisation rather than a measured property of the published objective. Measured on our tokenized corpus, supervised-token mass is 97.7% $\mathcal{L}_{\text{gen}}$ / 1.13% $\mathcal{L}_{\text{pos}}$ / 1.13% $\mathcal{L}_{\text{neg}}$, because a `gen` target is a whole program (196.7 tokens on average) while a `pos`/`neg` target is 3.0 tokens. Restoring `neg` to parity moves the auxiliary share from 2.26% to $\approx$2.6%, but that bound holds only under our target format. The reading it does not cover is an auxiliary target that *justifies* its verdict at length, which would carry comparable token mass and could quote the original program. We cannot exclude it. We note only that it would relocate the recovery from the objective to the supervision, which is this paper's thesis and contradicts the original's claim to need no reverse examples.

Three deliberate deviations. Our negatives are the *obfuscated* variant with one operator mutated and are execution-verified to differ from their parent, rather than clean programs of different semantics; under the original construction the question "is program B obfuscated?" predicts the label perfectly, so a model can score well on the auxiliary terms without comparing semantics at all. Appendix C runs the paper-literal construction rather than resting on that argument. Second, we pair the pools, keeping only `(program, condition)` keys with both a YES and a NO built from the same program, which downsamples `pos` to `neg` and is why both sit at $0.76\times$ `gen` (7 383 / 5 611 / 5 611); execution verification explains the `neg` shortfall, pairing explains the `pos` one. Third, we add "output only the code" to the generation prompt, without which the CodeBLEU comparison partly measures how much prose a model wrapped around its answer.

The cross-pass determinism floor. Evaluating the untuned `base` arm in five passes over a byte-identical 300-program set gives strict reverse rates of $2.9/2.9/2.9/2.7/2.7\%$. The program sets are identical in all five, and the two passes carrying the same arm list are bit-identical on all 1500 generations; the others differ on 6–8% of generations and flip 4–8 graded trials. The cause is therefore batch composition rather than corpus drift or a scorer change. The inference engine batches continuously, so which arms share a pass changes reduction order and occasionally an argmax. We treat ±0.3 pp as the resolution of any cross-pass comparison. It bounds nothing in the body, whose effects are 30 points, but it is the reason Appendix N carries an untuned anchor row.

C The paper-literal negative construction

Our `cft` arm builds negatives by mutating the *obfuscated* variant, so positives and negatives are equally obfuscated. The original builds them from clean programs of different semantics, which makes "is program B obfuscated?" a perfect predictor of the label. We argued in Appendix B that this shortcut makes the published construction unusable as a test of semantic comparison. The objection to that argument is that the shortcut may be the the operative factor, since a model that must represent obfuscated-ness explicitly has done part of the work of undoing it. So we built the paper-literal pools and trained the arm.

`cftclean` is `cft` with negatives mutated from the clean parent instead, identical in every other respect, at 1.5B and seed 17. It reaches 0.5% strict reverse success against 0.2% for both `cft` and `sft`, a difference of $+0.3$ pp $[-0.1, +0.7]$ against either, and all three sit 2.3 to 2.6 points *below* the untouched model. So the shortcut is not the the operative factor, and our deviation is not what produced the null. Under the original's own negative construction, carrying *more* auxiliary data than our main arm (6 058 against 5 611 per pool) and reaching an almost identical training loss (0.354 against 0.354), the objective still recovers nothing. The two arms therefore differ in what they were shown rather than in how well they fitted it. All four arms were scored in one evaluation pass, which matters because the cross-pass floor of Appendix B is wider than the contrast.

D The reverse-data dose response

The curve rises steeply and then flattens. Replacing 5% of the forward pairs recovers $+25.7$ pp $[+23.9, +27.7]$ over forward-only training, which is 85% of what a 50% share recovers; the remaining climb is $+4.5$ pp $[+3.2, +5.8]$ and has flattened by a quarter share, where 50% against 25% is $+1.1$ pp $[+0.0, +2.1]$ with an interval touching zero.

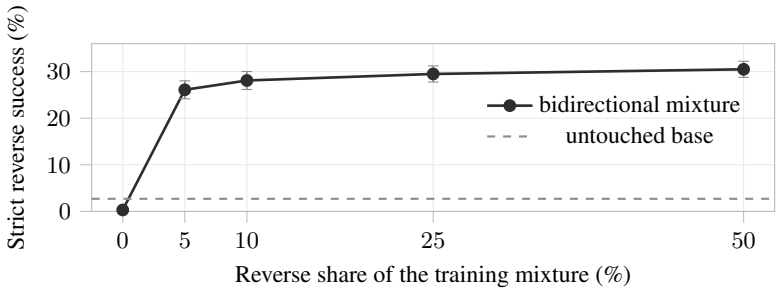

Figure 2: **A little reverse data does almost all of the work.** Strict reverse success against the reverse share of the training mixture, Qwen2.5-Coder-1.5B, 300 programs, 1 500 graded trials per point. The zero-dose point is forward-only SFT. Replacing just 5% of the forward pairs with their own reverse recovers 26.1% against 0.3% for forward-only — 85% of what a 50% share recovers. The remaining climb is real but small (+4.5 pp [+3.2, +5.8] from 5% to 50%) and has flattened by 25% (+1.1 pp [+0.0, +2.1] from 25% to 50%). Bars are 95% cluster bootstraps by program; because the design is paired, differences are quoted from the contrast table rather than read off the bars.

E Limitations, in full

Language and corpus. The original study is Java over 10 000 CodeNet programs, while ours is Python over 7 383 forward instances per epoch plus a JavaScript replication of the four load-bearing arms at 1.5B. Two languages is evidence that the effect is not an artifact of one, but neither is Java, so a Java-specific effect is not excluded. It is worth saying what such an effect would have to be. The equivalence-judgement terms would have to teach reverse deobfuscation in Java while teaching none of it in either Python or JavaScript, and to do so without the reversed data that accounts for the entire recovery in both languages we did test. We know of no mechanism with that shape, and the burden of proposing one falls on the claim that the objective is what works. **Scale and family.** Two sizes of one family, Qwen2.5-Coder 1.5B and 7B. The original's strongest CFT numbers come from commercial models we cannot fine-tune the same way, though our 7B result is at the tier of its open-source panel, and the 7B arms are single-seed. **One of the two general-capability probes overlaps the corpus**, as §5 sets out. MBPP+ is the held-out control reported beside it, and the HumanEval+ column against the untouched model should not be read on its own.

Transform coverage. String encryption, one of the original's three transforms and its hardest, is quarantined in our corpus as a held-out family for a separate study and is never trained on, so we report five conditions where the original reports three.

Auxiliary pool ratio. Both `pos` and `neg` are $0.76\times$ `gen`, for two different reasons set out in Appendix B. The token-share arithmetic there bounds the auxiliary signal at $\approx$2.6% of supervised tokens even at parity, so the shortfall cannot plausibly account for a 30-point gap, but we did not run the parity sweep, and that bound is conditional on our choice of auxiliary target format.

Pairing and seeds on the cost table. The HumanEval+ cells are a single greedy pass per arm and only aggregate counts were retained, so that column supports point estimates only. The MBPP+ cells retain per-task verdicts, so its contrasts are paired bootstraps over the 399 tasks confirmed by exact McNemar. Every 7B arm is single-seed, so none of these intervals covers training-seed variance.

F General-capability cost of the 1.5B dose ladder

arm	reverse share	HumanEval+ `plus`
`sft`	0%	.329
`mix5`	5%	.537
`mix10`	10%	.555
`mix25`	25%	.494
`mix50`	50%	.469
`rev`	100%	.585
`base`	n/a	.646

Qwen2.5-Coder-1.5B, 164 tasks, 0 scorer errors. The retained capability is *not* monotone in reverse share. `mix5` and `mix10` differ by 3 of 164 tasks, and `rev`, which has the largest reverse share, retains the most. We therefore draw no dose-response conclusion on this axis at 1.5B. The 7B comparison in §5 does not rest on these rows.

G Measured training budget (7B)

Table 5: Measured per-epoch training budget, 7B Python corpus, relative to forward-only SFT. Supervised tokens are the tokens the loss is actually computed on.

arm	instances	supervised tokens	sequence tokens ($\propto$ FLOPs)	steps
`fwd` *(reference)*	1.00×	1.00×	1.00×	1.00×
`fwd2x`	1.00×	1.00×	1.00×	2.00×
`mix50`	**1.00**×	**0.71**×	**1.05**×	**1.00**×
`rev`	1.00×	0.43×	1.09×	1.00×
`flip`	2.00×	1.43×	2.09×	2.00×
`cft`	2.52×	**1.02**×	**2.65**×	2.52×
`cftflip`	3.52×	1.45×	3.74×	3.52×

Sequence tokens set what a run costs and supervised tokens set what it teaches, so the two columns can move in opposite directions. `cft` is the case that matters, spending $2.65\times$ the compute of forward-only training to deliver $1.02\times$ the supervision.

H Scoring quantities

Table 6: The quantities used to score a trial. *Executes* is checked by running the program on the test inputs that ship with it. R returns 0 for a non-parsing output, since the readability of a non-program is undefined and 0 is the only resolution that cannot manufacture a success.

symbol	range	meaning
$\mathrm{parses}(\hat{y})$	$\{0,1\}$	the output is non-empty and parses in the target language
$S(a,b)$	$[0,1]$	CodeBLEU similarity between two programs [Ren et al., 2020]
$R(a)$	$[0,1]$	readability, a weighted sum of identifier meaningfulness (0.55), identifier length, line length and nesting depth (0.15 each)
$E(\hat{y})$	$\{0,1\}$	the output executes and reproduces x's outputs on every test input

Only S and R come from the published criterion. The parses guard and E are ours, and both can only lower a reported rate.

I The five obfuscating transforms

The split into identifier and structural families is what §5 turns on. Only the structural rewrites can be undone, because the identifier ones delete the original names rather than rearranging them.

J Strict reverse success by condition (7B)

arm	`L1b`	`L1r`	`L2`	`S1`	`S2`
`base`	.023	.110	.103	.330	.077
`sft`	.000	.000	.000	.000	.000
`cft`	.000	.000	.000	.003	.000
`fwd2x`	.000	.000	.000	.003	.000
`rev`	.003	.010	.017	**.817**	**.797**
`flip`	.017	.013	.023	**.827**	**.797**
`mix50`	.010	.007	.023	**.803**	**.797**

Table 7: The five obfuscating transforms. Each is applied on its own to the original function. We keep the short codes because the appendix tables report results per transform.

code	family	what it does
`L1b`	identifier	renames identifiers to misleading names, so `count` might become `buffer`
`L1r`	identifier	renames identifiers to random strings such as `v_a3f2`
`L2`	identifier	renames identifiers to single letters and strips type annotations
`S1`	structural	flattens control flow into a single dispatch loop driven by a state variable
`S2`	structural	inserts always-true tests and unreachable code

300 programs per cell, `simple` strategy, `e2_budget_qwen7b`. The capability is almost entirely structural, since the identifier conditions are at or below 2.3% for every arm, including reverse-only training, because adversarial renaming destroys information that no amount of reverse supervision can restore.

K Forward accuracy by condition (7B)

arm	`L1b`	`L1r`	`L2`	`S1`	`S2`
`base`	.950	.960	.963	.933	.837
`sft`	.970	.950	1.000	.933	1.000
`cft`	.973	.960	.997	.947	1.000
`fwd2x`	.967	.960	1.000	.917	1.000
`rev`	.940	.950	.957	.970	.820
`flip`	.970	.960	1.000	.927	.997
`mix50`	.973	.957	.990	.883	1.000

Execution-verified forward success. `flip` matches `sft` everywhere; bidirectionality is free in this direction too.

L Reverse prompting strategies (7B)

system	`simple`	`few_shot`	`cot`	`augmented`	range
`base`	13.2 [11.3, 15.3]	18.1 [15.7, 20.7]	**21.9** [19.4, 24.5]	21.4 [18.8, 24.1]	8.7
`sft`	0.0 [0.0, 0.0]	1.6 [1.0, 2.3]	**5.3** [4.2, 6.3]	5.1 [4.1, 6.1]	5.3
`cft`	0.1 [0.0, 0.2]	0.5 [0.1, 0.9]	**1.5** [0.9, 2.1]	1.2 [0.7, 1.7]	1.4
`rev`	**32.9** [31.5, 34.5]	32.5 [31.0, 34.0]	32.5 [31.0, 34.2]	32.9 [31.3, 34.4]	0.4
`flip`	**33.7** [32.1, 35.3]	32.6 [31.1, 34.1]	32.8 [31.2, 34.5]	32.6 [31.1, 34.1]	1.1
`mix50`	**32.8** [31.3, 34.4]	31.3 [29.6, 32.9]	30.3 [28.9, 31.9]	31.7 [30.0, 33.3]	2.5

Strict reverse success (%), cluster-bootstrap CIs by `program_id`; all six arms scored in one evaluation pass (`e7_strategies_qwen7b`), best strategy per arm in bold. Both forward-only arms at their best strategy fall below the untuned base at its worst. The bidirectional arms (lower block) are nearly flat across strategies while the untuned base spans 8.7 points, so capability installed by data does not depend on how it is elicited.

M What the reverse criterion certifies (7B, `simple`)

system	cond	paper %	strict %	id-rec (passes)	id-rec (all)	**lift**	floor
`base`	`L1b`	4.7	2.3	0.326	0.363	−0.037	0.354
`base`	`L1r`	19.0	11.0	0.428	0.423	+0.005	0.349
`base`	`L2`	18.7	10.3	0.437	0.420	+0.017	0.389
`base`	`S1`	64.7	33.0	0.895	0.906	−0.011	1.000
`base`	`S2`	9.3	7.7	0.750	0.878	−0.127	1.000
`flip`	`S1`	90.7	82.7	0.999	0.999	+0.001	1.000
`flip`	`S2`	79.7	79.7	1.000	0.999	+0.000	1.000
`mix50`	`S1`	90.3	80.3	0.994	0.994	−0.000	1.000
`mix50`	`S2`	79.7	79.7	0.999	0.999	−0.000	1.000

floor is the identifier recall of the obfuscated input itself, which is what echoing scores. *lift* is id-recall among criterion-passing outputs minus id-recall among all outputs; a lift near zero means the criterion certifies nothing about identifier recovery. The structural conditions have a floor of exactly 1.000, so identifier recall does not discriminate there at all.

N Seed replication (1.5B, strict reverse success %)

arm	seed 17	seed 42	Δ
`sft`	0.4	0.4	+0.0
`cft`	0.3	0.4	+0.1
`flip`	31.5	30.9	−0.6
`rev`	31.5	31.2	−0.3
`mix50`	30.6	30.0	−0.6
`flipsym`	30.5	30.8	+0.3
`base` *(untuned anchor)*	2.9	2.7	−0.2

Identical harness, program set, criteria and decoding; the adapter seed is the only difference. Columns come from two evaluation passes (`e1_qwen1.5b`, `e2_seeds_qwen1.5b`). The `base` row has no seed and is the same untouched model in both, so it calibrates the table. Roughly 0.2 pp of the movement in every other row is cross-pass noise rather than seed variance (§2). The two arms carrying the paper's null move by ≤ 0.1. `flipsym` trains `flip`'s mixture under a single shared system prompt and is a control against the two directions being separable merely because they arrived under two personas.

O JavaScript replication (1.5B, strict reverse success %)

arm	JavaScript	contrast vs. Python
`base`	3.8 [2.5, 5.3]	(Python 2.9)
`sft`	0.0 [0.0, 0.0]	`sft`−`base` −3.8 [-5.6, -2.4]
`cft`	0.4 [0.0, 0.8]	`cft`−`sft` +0.4 [+0.0, +0.8]
`mix50`	30.3 [27.4, 33.1]	`mix50`−`sft` +30.3 [+28.3, +32.5]
`flip`	30.4 [27.5, 33.2]	`flip`−`mix50` +0.1 [-0.2, +0.5]

167 programs, 835 graded trials per arm, same conditions and criteria as the Python corpus. Every qualitative conclusion holds. Forward-only tuning lands below the untouched model, the contrastive objective does not repair it, and the budget-matched bidirectional arm captures the whole effect. Here it is statistically indistinguishable from the doubled-data arm, whereas in Python the latter led by +0.7 pp.